An NVMe Key-Value namespace backed by RADOS

Kyle Bader

2026-06-05

Contents

Why this is more powerful than it first looks . . . 1
The mapping . . . 2
The 64 MB value ceiling (and why it's there) . . . 2
The fun part: allowlisted RADOS classes per namespace . . . 3
A real example: extract an embedding at the OSD . . . 3
What it looks like to set up . . . 4
Seeing it work end to end . . . 4
Closing . . . 6

What if a guest could speak to Ceph the way it speaks to a local NVMe drive — no client library, no config file, no keyring — and on the other side of that wire it got the full power of RADOS? Not a block device pretending to be an object store, but the actual key-value grain of RADOS, exposed as a first-class NVMe namespace.

That is what this prototype does. It teaches SPDK to present an **NVMe Key-Value (KV) namespace** whose backend is **librados**. A host — physical or virtual — attaches the namespace and issues KV commands (Store, Retrieve, Delete, Exist). Each one becomes a RADOS operation. The host never links librados, never sees a `ceph.conf`, never holds a cephx key. It just sees an NVMe device that happens to be infinitely large and lives in your cluster.

> **Status:** this is experimental work built on top of SPDK's existing host-side KV driver. It is not upstream in SPDK or Ceph. The point of this post is to show the shape of the idea and why the mapping is so natural.

Why this is more powerful than it first looks

NVMe-oF already decouples *where the drive is* from *where the workload runs*. The KV-over-RADOS target inherits that, and it means the same software can sit almost anywhere on the path between an application and its data:

- **On the storage host** — the SPDK target runs next to the OSDs, presenting KV namespaces out to the network.
- **On the hypervisor** — the target runs on the compute node and presents KV namespaces *up* into guests over `vfio-user`, so a VM sees a KV "drive" with no Ceph client inside the guest at all.
- **In a DPU / SmartNIC** — on either the storage side or the hypervisor side, the target runs on the DPU's Arm cores, completely off the host x86 and out of the guest's trust domain.

The guest sees NVMe; the DPU does RADOS.

In every one of those placements the guest contract is identical: *attach a key-value namespace, get the raw power of RADOS behind it.* The KV namespace is the portable unit. You can move where it terminates without touching the guest.

The mapping

The reason this is clean rather than clever is that the NVMe tenancy hierarchy lines up almost one-to-one with RADOS primitives. We did not have to invent a translation layer; we just had to *name* the correspondence.

NVMe-oF concept	RADOS concept
Subsystem	**Pool**
Namespace	**Namespace (in pool)**
Key (1–16 bytes)	**Object name (oid)**
Value	**Object data**

- **Subsystem -> RADOS pool.** Each NVMe-oF subsystem that uses the librados backend maps to one distinct pool. Pools give you strong isolation and independent placement, replication, and quota policy — so each subsystem carries its own durability and performance contract. (Pools are operator-provisioned; the target does not create them. Pools aren't free in Ceph, so this is the right granularity for "a tenant / a service," not for millions of namespaces.)
- **Namespace -> RADOS namespace.** Each KV namespace inside a subsystem maps to one RADOS *namespace* within that subsystem's pool, set on the ioctx with `rados_ioctx_set_namespace`. RADOS namespaces are nearly free, so this is the cheap, fine-grained tenancy axis. Two guests can each own a namespace in the same pool and their key spaces never collide — identical keys in different NVMe namespaces resolve to different objects.
- **Key -> oid, Value -> object data.** A KV pair is exactly one RADOS object. Store is `aio_write_full`, Retrieve is `aio_read`, Exist is `aio_stat`, Delete is `aio_remove`. No omap index funneling every key through one hot object, no striping layer — one key, one object, distributed across PGs and OSDs the way RADOS already does best.

So when a guest does:

```
Store(namespace=1, key="user/42/avatar", value=<image bytes>)
```

what lands in Ceph is an object named after that key, in the RADOS namespace bound to NVMe namespace 1, in the pool bound to that subsystem. It is RADOS all the way down — the same objects your other Ceph clients can see, governed by the same CRUSH rules, scrubbed and replicated like everything else.

The 64 MB value ceiling (and why it's there)

Because a Value *is* a single RADOS object's data, the value size is bounded by the RADOS object-size limit. The backend caps a KV value at **64 MB** — a conservative cap under the default `osd_max_object_size` of 128 MiB. A Store that exceeds the cap is *rejected*, not silently split: there is no `libradosstriper` in the path, so a value is never smeared across multiple objects.

This isn't a wart, it's the contract. The maximum is advertised to the host in the KV Identify namespace data (the `kvvml` / max-value-length field), so a well-behaved initiator knows the ceiling before it ever issues a Store. Keys are unaffected — NVMe KV keys are <=16 bytes by spec. If you need values bigger than 64 MB, that's a block namespace or a striped object, not this.

The fun part: allowlisted RADOS classes per namespace

Here is where a KV namespace stops being "a place to put bytes" and becomes a place to *run code next to your bytes*.

RADOS has always had **object classes** (`cls`) — server-side plugins that execute on the OSD that owns an object, with the data already in cache and zero network round-trips to read it. This prototype adds a vendor NVMe command, **KV Exec** (opcode `0x83`), that maps onto `rados_exec`. A guest issues KV Exec against a key with a small **op-ID** and an input blob; the named class method runs on the OSD, against the object backing that key, and returns an output blob. Compute moves to the data instead of the data moving to compute — over a plain NVMe command.

The obvious objection is "you've just let a guest run arbitrary code on my OSDs." We didn't. Two design choices close that door:

1. **The data path never names a class or method.** The KV Exec command carries an integer **op-ID**, never a `class/method` string. There is nothing for a malicious initiator to inject — arbitrary `cls/method` strings simply cannot cross the wire.
2. **A per-namespace allowlist, default-deny.** A privileged control plane binds each op-ID to a concrete `(class, method)` *for one namespace only*. Op-ID 1 in namespace A might mean `kvtest:echo`; the same op-ID 1 in namespace B might mean nothing at all, or something else entirely. An unknown op-ID is rejected. The trust decision lives entirely in the control plane, scoped to the same RADOS-namespace tenancy boundary as the data — managed out-of-band by a small gRPC sidecar that pushes the allowlist into the target over SPDK's JSON-RPC, so gRPC and protobuf never enter the data-plane core.

In the working end-to-end test, op-ID 1 binds to `kvtest:echo` (returns the input) and op-ID 2 to `kvtest:upcase` (reads the stored object value server-side and returns it uppercased). Trivial classes — but they prove the whole path: a guest stored a value, asked the OSD to transform it in place, and got the result back without the value ever traversing the network to be read.

A real example: extract an embedding at the OSD

`upcase` is a toy. Replace it with something an AI pipeline actually wants.

Imagine a namespace whose values are images, and a custom object class `embed:forward` that loads a small quantized model and runs a forward pass over the object's bytes, returning a fixed-length embedding vector. An operator allowlists it for exactly that namespace:

```
# control plane (sidecar), keyed by (subsystem, namespace):
SetAllowlist(subsystem="nqn...:images", namespace=1, bindings=[
    (op_id=7, binding="embed:forward"),
])
```

Now the guest's data path is just NVMe:

```
Store(ns=1, key="photo/8842", value=<jpeg bytes>)   # 1 RADOS object
```

```
KV-Exec(ns=1, key="photo/8842", op_id=7, in=<params>)
        -> returns <float32[768] embedding>
```

The image never leaves the OSD to be embedded. No GPU box pulls 4 MB across the network per image; the OSD that already holds the object runs the forward pass where the bytes live and ships back a few kilobytes of vector. The guest needs no model, no librados, no GPU, and no permission to run anything except op-ID 7 on its own namespace. Index-time embedding, content hashing, format transcoding, thumbnailing, server-side filtering — all of it becomes "store the value, exec an allowlisted op-ID against it."

That is the thesis in one line: **a KV namespace is a portable handle to RADOS — its objects, its tenancy, and now its server-side compute — delivered to any host over a standard NVMe command set.**

What it looks like to set up

Standing up a librados-backed KV namespace is a handful of SPDK RPCs against a running `nvmf_tgt`:

```
# Register the Ceph cluster connection (cephx + ceph.conf stay target-side)
rpc.py kvdev_rados_register_cluster ceph0 \
    --user admin --config-file /etc/ceph/ceph.conf --key-file /etc/ceph/keyring

# Create a kvdev bound to pool 'kvpool', RADOS namespace 'kvns'
rpc.py kvdev_rados_create KvRados0 ceph0 kvpool --namespace kvns

# Expose it as a KV namespace under a subsystem
rpc.py nvmf_create_subsystem nqn.2026-06.io.spdk:kv0 -a
rpc.py nvmf_subsystem_add_kv_ns nqn.2026-06.io.spdk:kv0 KvRados0
rpc.py nvmf_subsystem_add_listener nqn.2026-06.io.spdk:kv0 -t VFIOUSER ...

# (optional) allowlist a cls op for this namespace
rpc.py nvmf_ns_set_kv_exec_allowlist nqn.2026-06.io.spdk:kv0 1 \
    --op 7:embed:forward
```

On creation the target logs the negotiated cap, e.g. `max_value_len=67108864` — exactly 64 MiB. Attach the namespace from a guest and an `fio` job using the SPDK NVMe engine round-trips Store/Retrieve with `verify=crc32c` cleanly: the writes are RADOS objects, the reads come back byte-for-byte, and `rados ls -p kvpool --namespace kvns` shows them sitting in your cluster.

Seeing it work end to end

This isn't a thought experiment — it runs today. The repository ships a test that brings up an `nvmf_tgt`, wires a librados-backed KV namespace to a vstart Ceph cluster (pool `kvpool`, RADOS namespace `kvns`), exposes it over `vfio-user`, and drives it with `fio`'s SPDK NVMe engine doing `verify=crc32c` round-trips:

```
$ sudo CEPH_CONF=/mnt/ceph/build/ceph.conf RADOS_BIN=/mnt/ceph/build/bin/rados \
       CONFIG_FIO_SOURCE_DIR=/mnt/ceph/build/src/fio \
       bash test/nvmf/kv_rados/kv_rados_fio.sh
```

```
[...] Starting SPDK v26.09-pre git sha1 ef25c6429 / DPDK 26.03.0 initialization...

# register the Ceph cluster (cephx + ceph.conf stay target-side)
rpc.py ... kvdev_rados_register_cluster ceph0 --user admin \
           --config-file .../ceph.conf --key-file .../keyring
kvdev_rados.c:380: *NOTICE*: Registered rados cluster 'ceph0'

# bind a kvdev to pool 'kvpool', RADOS namespace 'kvns'
rpc.py ... kvdev_rados_create KvRadosFio0 ceph0 kvpool --namespace kvns
kvdev_rados.c:1175: *NOTICE*: Created rados kvdev 'KvRadosFio0'
        (pool=kvpool, namespace=kvns, max_value_len=67108864)

# expose it as a KV namespace and attach fio's SPDK NVMe engine
rpc.py ... nvmf_subsystem_add_kv_ns nqn.2026-06.io.spdk:kv-rados-fio0 KvRadosFio0
```

The negotiated cap in that log — `max_value_len=67108864` — is exactly 64 MiB, the value ceiling advertised to the host. The `fio` run then writes and reads back 1024 blocks of 4 KiB and the `crc32c` verify passes clean:

```
kv-verify: (groupid=0, jobs=1): err= 0: pid=2275383
  read : IOPS=9061, BW=35.4MiB/s (37.1MB/s)(4096KiB/113msec)
  write: IOPS=4853, BW=19.0MiB/s (19.9MB/s)(4096KiB/211msec)
  issued rwts: total=1024,1024,0,0   short=0,0,0,0   dropped=0,0,0,0

kv_rados_fio: PASS (fio Store/Retrieve+verify over librados)
```

Now the proof that those NVMe Stores really *are* RADOS objects — ask Ceph directly. Each Store landed as one object, named by the (encoded) key, inside exactly the pool and RADOS namespace we bound:

```
$ rados -p kvpool --namespace kvns ls | wc -l
1024                                  # one object per fio block - nothing lost

$ rados -p kvpool --namespace kvns ls | sort | head
0000000000000000                      # key -> oid: the encoded NVMe KV key
0000000000000001
0000000000000002
0000000000000003
...

$ rados -p kvpool --namespace kvns stat 00000000000003f0
kvpool/00000000000003f0 mtime 2026-06-05T15:59:36 ..., size 4096   # value -> object data (4 K

$ rados -p kvpool ls | wc -l          # the pool's DEFAULT namespace:
5                                     # the fio keys are NOT here - namespace isolation holds
```

There it is, top to bottom: a guest issued NVMe KV Stores over a standard command set; 1024 of them became 1024 RADOS objects, named by key, sized by value, living in the pool and namespace the subsystem mapped to — and visible to every other Ceph client in the cluster. The keys are confined to `kvns`; the pool's default namespace never sees them. The mapping isn't a metaphor; it's

what's on disk.

Closing

None of the individual pieces are exotic. NVMe-oF gives you a transport and a namespace abstraction. RADOS gives you a distributed object store with server-side compute. The work here is mostly in *recognizing that they line up* — subsystem to pool, namespace to namespace, key to object, value to object data — and then being disciplined about the one genuinely dangerous edge, server-side execution, by making it default-deny, op-ID-indirected, and scoped to the same tenancy boundary as the data.

The payoff is a single portable primitive — the KV namespace — that you can terminate on the storage host, the hypervisor, or a DPU, and behind which a guest gets the raw power of RADOS without ever knowing Ceph is there.